

Lesson 5: Data Cleaning and Validation

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Spot the dirty data
I Do	10–15 min	Missing data, types, duplicates, outliers, regex
We Do	15–20 min	Clean a messy DataFrame together
You Do	5–10 min	Validation challenge
Wrap-Up	5 min	Assignment preview & capstone kickoff

Warm-Up (5 min)

Spot the Problems

```
data = {  
    "Name": ["Alice", "  bob ", None, "CHARLIE"],  
    "Age": ["25", "thirty", "35", "-5"],  
    "Salary": [50000, "n/a", 70000, 80000],  
    "Hire Date": ["2023-01-15", "01/20/2023", None, "not a date"]  
}
```

How many data quality issues can you find? List them in the chat.

I Do: Why Clean Data?

Garbage in → Garbage out.

Real-world data is messy. Before you can analyze it, you need to:

Problem	Consequence
Missing values	Calculations fail or give wrong results
Wrong types	<code>"25" + 1</code> throws an error
Duplicates	Inflated counts and totals
Outliers	Skewed averages and charts
Inconsistent text	<code>"NYC" ≠ "New York"</code> in a groupby

Rule of thumb: Always keep an untouched copy of raw data before cleaning.

I Do: Handling Missing Data

Three key methods:

```
# Find missing values
df[df.isnull().any(axis=1)]

# Drop rows with ANY missing value
df_clean = df.dropna()

# Fill missing values with specific strategies
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Name"] = df["Name"].fillna("Unknown")
```

When to drop vs. fill?

- **Drop** when few rows are affected and data is plentiful
- **Fill** when you can't afford to lose rows (use mean, median, or a constant)

I Do: Fixing Data Types

Columns often load as the **wrong type** — especially from CSV:

```
# Convert string numbers to actual numbers
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

# Convert strings to integers
df["Age"] = df["Age"].astype(int)

# Parse dates
df["Hire Date"] = pd.to_datetime(df["Hire Date"], errors="coerce")
```

`errors="coerce"` turns invalid values into `NaN` instead of crashing.

Always check `df.dtypes` after loading — if a number column says `object`, it's probably strings.

I Do: Removing Duplicates

```
# See which rows are duplicates
df.duplicated()

# Count duplicates
df.duplicated().sum()

# Remove duplicates (keeps first occurrence)
df_clean = df.drop_duplicates()

# Remove duplicates based on specific columns
df_clean = df.drop_duplicates(subset=["Name", "Age"])
```

Why duplicates happen:

- Data merged from multiple sources
- Form submitted twice
- Copy-paste errors

I Do: Validating Ranges & Handling Outliers

Replace out-of-range values with NaN:

```
import numpy as np

# Ages must be 18–65
df["Age"] = df["Age"].apply(
    lambda x: x if 18 <= x <= 65 else np.nan
)

# Fill the NaNs with the median
df["Age"] = df["Age"].fillna(df["Age"].median())
```


I Do: Validating Ranges & Handling Outliers (Cont.)

For outliers (e.g., Age > 100 or < 0):

```
df["Age"] = df["Age"].apply(  
    lambda x: df["Age"].median() if x > 100 or x < 0 else x  
)
```

Domain knowledge matters — an age of 95 might be valid; a salary of -\$500 probably isn't.

I Do: Text Standardization

Inconsistent text breaks grouping and matching:

```
# Lowercase and strip whitespace
df["Name"] = df["Name"].str.lower().str.strip()

# Replace abbreviations
df["City"] = df["City"].replace({
    "NYC": "New York",
    "LA": "Los Angeles"
})
```

I Do: Text Standardization (Cont.)

`map()` vs `replace()`:

Method	Unmapped values become...
<code>map()</code>	<code>NaN</code> (dangerous!)
<code>replace()</code>	Unchanged (safer)

Use `replace()` when you only want to fix specific values.

I Do: Regex Basics for Cleaning

Regular expressions help clean messy text patterns:

```
# Remove non-digit characters from phone numbers
df["Phone"] = df["Phone"].str.replace(r"\D", "", regex=True)

# Extract domain from email
df["Domain"] = df["Email"].str.extract(r"@" + r"(\w[\w\.-]+)")

# Find rows containing "ship" (case-insensitive)
shipped = df[df["Status"].str.contains("ship", case=False)]
```

I Do: Regex Basics for Cleaning (Cont.)

Common patterns:

Pattern	Matches
<code>\d</code>	Any digit
<code>\D</code>	Any non-digit
<code>\w</code>	Word character (letter/digit/_)
<code>. * ?</code>	Any characters (non-greedy)

We Do: Let's Clean a Messy DataFrame

Here's our dirty data:

```
import pandas as pd
import numpy as np

data = {
    "Name": ["  Alice ", "BOB", None, "charlie", "Alice"],
    "Age": [25, "unknown", 35, -5, 25],
    "Salary": [50000, 60000, "n/a", 80000, 50000],
    "City": ["NYC", "LA", "Chicago", "NYC", "NYC"]
}
df = pd.DataFrame(data)
print(df)
```

How many issues can we spot?

We Do: Step 1 — Fix Text

```
# Standardize names: strip whitespace, lowercase
df["Name"] = df["Name"].str.strip().str.lower()

# Fill missing names
df["Name"] = df["Name"].fillna("unknown")

# Standardize city abbreviations
df["City"] = df["City"].replace({
    "NYC": "New York",
    "LA": "Los Angeles"
})
print(df)
```

Check: Are all names lowercase and trimmed now?

We Do: Step 2 — Fix Types and Missing Values

```
# Convert Age to numeric (invalid → NaN)
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

# Convert Salary (replace placeholder first)
df["Salary"] = df["Salary"].replace("n/a", np.nan)
df["Salary"] = pd.to_numeric(df["Salary"], errors="coerce")

# Fill missing values
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(df["Salary"].median())

print(df.dtypes)
print(df)
```

Why replace "n/a" before `to_numeric()` ?

`to_numeric` with `errors="coerce"` would also catch it, but explicit replacement is clearer and self-documenting.

We Do: Step 3 — Validate and Deduplicate

```
# Validate age range (must be 0–120)
df["Age"] = df["Age"].apply(
    lambda x: x if 0 <= x <= 120 else np.nan
)
df["Age"] = df["Age"].fillna(df["Age"].median())

# Remove duplicates
print(f"Before: {len(df)} rows")
df = df.drop_duplicates()
print(f"After: {len(df)} rows")

print(df)
```

Discussion: Why did we validate *after* converting types?

You can't check `if 0 <= x <= 120` on a string — the column must be numeric first.

We Do: The Cleaning Pipeline

Here's the pattern we just followed:

Raw Data

- ↓ 1. Standardize text (strip, lowercase)
- ↓ 2. Fill missing text values
- ↓ 3. Convert types (to_numeric, to_datetime)
- ↓ 4. Fill missing numeric values (mean/median)
- ↓ 5. Validate ranges
- ↓ 6. Remove duplicates

Clean Data ✓

This pipeline works for most datasets. The assignment follows this exact order.

You Do: Validation Challenge (5 min)

Given this DataFrame:

```
df = pd.DataFrame({  
    "Product": ["Widget", "Gadget", "Widget", "Doohickey"],  
    "Price": [10.99, -5.00, 10.99, 1500.00],  
    "Rating": [4.5, 3.2, 4.5, 6.1]  
})
```

Write code to:

1. Remove duplicate rows
2. Replace negative prices with `NaN`, then fill with the median
3. Replace ratings > 5.0 with `NaN`, then fill with the median

Assignment Preview

This week's assignment is on **Kaggle** and covers 10 tasks:

Tasks	Focus
1	Handling missing data (dropna, fillna)
2	Data type conversion (dates with errors)
3	Validating age ranges
4	Removing duplicates & outliers
5	Standardizing text (names, cities)
6	Encoding categorical variables
7	Mini-project: consolidating 4 messy CSV files
8	Regex for validation and extraction

Assignment Tips

Tasks 1–5: Follow the cleaning pipeline from our We Do — same order, same techniques.

Task 6 (encoding): Two approaches to learn:

- `map()` for label encoding (Red → 1, Blue → 2)
- `pd.get_dummies()` for one-hot encoding (separate column per category)

Task 7 (messy files): The `thefuzz` library handles approximate string matching. Don't worry about understanding every line — focus on the *pattern*.

Task 10 (capstone): Pick your dataset this week and start exploring it. Just get it loaded and run `info()`.

Capstone Project Kickoff

Starting this week, you'll build a **data pipeline project** on Kaggle.

Dataset options:

- Global Superstore (retail sales)
- TMDB 5000 Movies (film data)
- Life Expectancy (WHO health data)
- Seattle Airbnb (listing data)

This week's goal: Create your Kaggle notebook, pick a dataset, load it, and start cleaning.

You'll add to this project each week as you learn new techniques.

Wrap-Up

Today we covered:

- Handling missing data (`dropna`, `fillna`)
- Fixing data types (`to_numeric`, `to_datetime`, `astype`)
- Validating ranges and handling outliers
- Removing duplicates (`drop_duplicates`)
- Standardizing text and intro to regex
- The cleaning pipeline pattern

Quick check: What's the first thing you should do after loading a new dataset?

Getting Help This Week

- Follow the **cleaning pipeline** order — it prevents cascading errors
- Use the **Python interactive shell** to test cleaning steps one at a time
- **Task 7** is a mini-project — budget extra time for it
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for help with regex or the capstone

Next week: Data Wrangling and Aggregation — groupby, pivot tables, and merging!

Clean Data, Clear Insights! 🖌️

Garbage in, garbage out — but clean data in, insights out.

See you next session!